Quantum Variational Algorithms on Adiabatic Quantum Computing Devices

Philipp Hanussek

August 13, 2026

Glossary

CEM Conditional Expectation Matching. 6, 10, 14

CG conjugate gradient. 19

JIT just-in-time. 20, 21

KPO Kerr-nonlinear parametric oscillators. 5

LSB Langevin Simulated Bifurcation. 6, 9, 10, 12

MCMC Markov chain Monte Carlo. 6

QUBO quadratic unconstrained binary optimization. 3

RBM restricted Boltzmann machine. 8–13, 15, 16, 18, 19

RMSE root mean squared error. 14

SA simulated annealing. 7, 9, 10

SR stochastic reconfiguration. 18, 19

SRBM Semi-Restricted Boltzmann Machine. 6

TFIM Transverse Field Ising Model. 4, 9, 11, 13, 15, 19

TTE time to ϵ. 9, 20

TTS time to solution. 3

VQE Variational Quantum Eigensolver. 7

1 Overview

Since the development of programmable quantum processors, many claims regarding quantum advantage have been made. Often, these advantage claims did not last long. In this work, we focus exclusively on the D-Wave Quantum Annealer, which leverages quantum technology to solve quadratic unconstrained binary optimization (QUBO) problems. D-Wave is among the first companies to commercially sell quantum computers. In 2025, researchers from D-Wave claimed to have solved a scientifically relevant problem faster using quantum technology than it would have been possible on classical devices. However, it is currently an open question whether the D-Wave Quantum Annealer is able to solve other problems well, and possibly faster than conventional computers. In this work, we assess D-Wave's ability to sample from a probability distribution, to approximate the ground state of different statistical mechanical models using variational algorithms.

1.1 Previous work & motivation

We give an overview of the current research on quantum computing devices and present our motivation for using variational algorithms to simulate quantum systems.

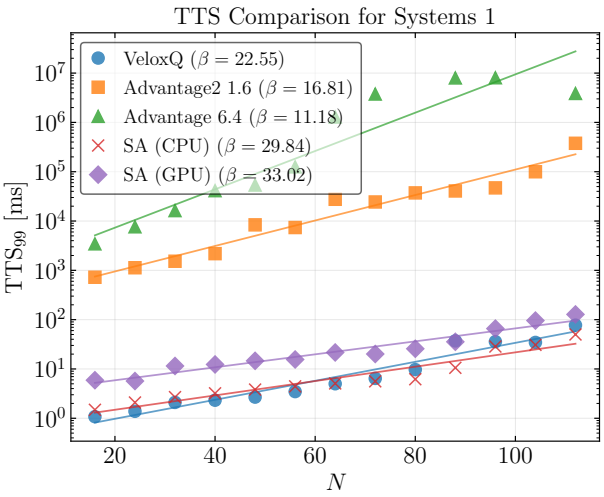
1.1.1 Quantum Annealing Performance

The D-Wave Quantum Annealer's performance scaling for finding the ground state (corresponding to the minimum of the quadratic binary cost function) has been extensively studied, in the area of cryptography and physical simulation. In general, classical methods exhibit faster solving times and better scaling. We briefly report on previous work in the field of simulating physical systems using quantum annealing hardware. In [1], we used a method presented in [2] to encode the solution to the wave function of different Hamiltonians into a QUBO problem. These instances were then solved on different physical and physics-inspired solvers. We evaluate the scaling of the samplers using the time to solution (TTS) metric, which corresponds to the computational time needed to solve the optimisation problem with a success probability of at least P_{target}. The TTS is defined as

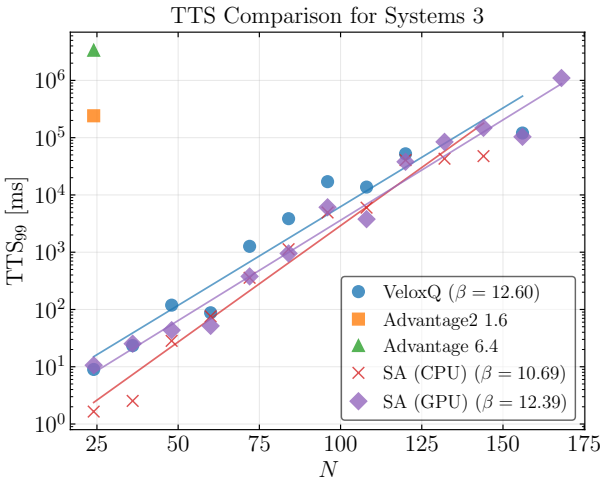
$$\text{TTS} = \frac{\ln(1 - P_{\text{target}})}{\ln(1 - P_{\text{success}})} T_r, \tag{1}$$

where P_{success} is the probability of obtaining the ground state in a single run, and T_r denotes the runtime of a single run.

It is important to note that current D-Wave devices exhibit limited connectivity. As a consequence, most problem instances need to be adapted in an additional *embedding* step, increasing the number of physical qubits required. We compare two problem formulations, one advantageous for the quantum annealer, as no extra embedding step is required, and a second one after embedding. Classical devices dominate in both cases (see Figure 1).



(a) D-Wave native formulation. This corresponds to a single-qubit Y rotation, governed by $H_1 = \frac{\pi}{2}\sigma_y$.



(b) Non-native formulation. This corresponds to a single-qubit Rabi oscillation driven along y, described by $H_3 = \pi\left(\frac{3}{4}\mathbb{1} - \frac{1}{4}\sigma_y\right)$.

Figure 1: Previous work on sampling the exact ground state to simulate quantum systems. TTS99 vs. N: exponential scaling $\propto \exp(N/\beta)$. Classical solvers dominate for $N \gtrsim 100$; missing points indicate zero success probability.

1.1.2 Motivation

As previously shown, results returned by the quantum annealer are inherently noisy. This makes it difficult to sample the *exact* ground state of the submitted Hamiltonian. In this work, we therefore assess the device's ability to act as a Gibbs sampler and sample from the Boltzmann distribution. It has been shown earlier that the D-Wave device can in fact be used as a Gibbs sampler[3, 4]. Here, we systematically evaluate the quality and time of these samples on computationally hard problems, which allows us to rigorously compare the quantum annealer to classical physics inspired algorithms.

2 Methods

In this section we briefly outline the simulated physical systems as well as the main computational and algorithmic methods used in this work.

2.1 Quantum Ising Models

The Ising model consists of discrete spins arranged on a lattice, interacting via nearest-neighbour couplings. In its quantum extension, the Transverse Field Ising Model (TFIM), a transverse magnetic field is added, introducing quantum fluctuations through non-commuting terms in the Hamiltonian.

2.1.1 Transverse Field Ising Model

The Hamiltonian for the one-dimensional TFIM is given by

$$H = -J \sum_i \sigma_i^z \sigma_{i+1}^z - h \sum_i \sigma_i^x \tag{2}$$

where $J > 0$ is the ferromagnetic coupling strength, $h \geq 0$ is the transverse magnetic field strength, and periodic boundary conditions are imposed on the chain. The 1D TFIM serves as a benchmark for our models, as it is exactly solvable via the Jordan–Wigner transformation. Despite its simplicity, it displays non-trivial quantum behavior, including a quantum phase transition and quantum fluctuations.

2.2 Heisenberg Models

The Heisenberg model describes spins on lattice sites interacting via exchange coupling [5]:

$$H = \sum_{\langle i,j \rangle} J_{ij}\, \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_j \tag{3}$$

where $\boldsymbol{\sigma}_i = (\sigma_i^x, \sigma_i^y, \sigma_i^z)$ are the Pauli spin operators, J_{ij} is the exchange coupling between sites i and j, and $\langle i, j \rangle$ denotes nearest-neighbour pairs. Spin operators are represented by Pauli matrices (eigenvalues ± 1) rather than spin-$\frac{1}{2}$ operators (eigenvalues $\pm\frac{1}{2}$) throughout this work. In contrast to the TFIM, the Heisenberg model couples all three spin components.

2.2.1 Frustrated J_1-J_2 Heisenberg Chain

The frustrated J_1-J_2 Heisenberg chain extends the Heisenberg model with a next-nearest-neighbour exchange term:

$$H = J_1 \sum_i \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_{i+1} + J_2 \sum_i \boldsymbol{\sigma}_i \cdot \boldsymbol{\sigma}_{i+2}, \tag{4}$$

where J_1 and J_2 are the nearest- and next-nearest-neighbour coupling strengths. The competing interactions frustrate the system, preventing simultaneous minimisation of both exchange terms. Setting $J_2 = 0$ recovers the standard Heisenberg chain, while increasing J_2/J_1 drives the system into a more frustrated regime that poses a more stringent test for variational methods.

2.3 Quantum (inspired) solvers

Here we present two solver types: First, a special quantum machine designed to find the ground state of an Ising Hamiltonian, and second, a classical quantum inspired algorithm.

2.3.1 Adiabatic Quantum Computing

Adiabatic Quantum Computing is based on the adiabatic theorem, which states: *A physical system remains in its instantaneous eigenstate if a given perturbation is acting on it slowly enough and if there is a gap between the eigenvalue and the rest of the Hamiltonian's spectrum.* In this work we focus on the D-Wave Quantum Annealer, which acts according to the adiabatic theorem in the following way: In the beginning of the annealing process, the device is initiated in the ground state of an initial driver Hamiltonian H_D that can be easily constructed:

$$H_D = -\sum_{i=1}^{N} \sigma_x^{(i)} \text{ with ground state } |+\rangle^{\otimes N} \tag{5}$$

where σ_x and σ_z are the Pauli matrices acting on qubit i. The problem Hamiltonian is defined as

$$H_P = \sum_{i=1}^{N} h_i \sigma_z^{(i)} + \sum_{i=1}^{N} \sum_{j=1}^{i-1} J_{ij} \sigma_z^{(i)} \otimes \sigma_z^{(j)}. \tag{6}$$

During the annealing process, the system is slowly brought from H_D to H_P according to the so-called annealing schedules $A(s)$ and $B(s)$:

$$H(s) = A(s)H_D + B(s)H_P = -A(s) \sum_{i=1}^{N} \sigma_x^{(i)} + B(s) \sum_{i=1}^{N} h_i \sigma_z^{(i)} + B(s) \sum_{i=1}^{N} \sum_{j=1}^{i-1} J_{ij}\, \sigma_z^{(i)} \otimes \sigma_z^{(j)}, \tag{7}$$

where $s \in [0, 1]$ is the normalized time. $A(s)$ decreases the driving terms in H_D as s increases, and $B(s)$ activates the terms of the problem Hamiltonian H_P. At the end of the evolution, if the process happens slow enough, and assuming a noiseless environment, the system will be in the ground state of the problem Hamiltonian according to the adiabatic theorem. It is important to note that the qubits in the D-Wave Quantum Annealer are not *all-to-all* connected. The qubits are arranged in an architecture dependent grid. In the Pegasus topology, one qubit couples on average to 15 other qubits, while Zephyr qubits couple to 20 other qubits. Therefore, most problems require an additional embedding step, in which multiple physical qubits are *chained* together to form one logical qubit. For a toy-problem example of embedding a triangle shaped connectivity pattern on a QPU, see Figure 2c. There exist three different types of couplers, connecting physical qubits:

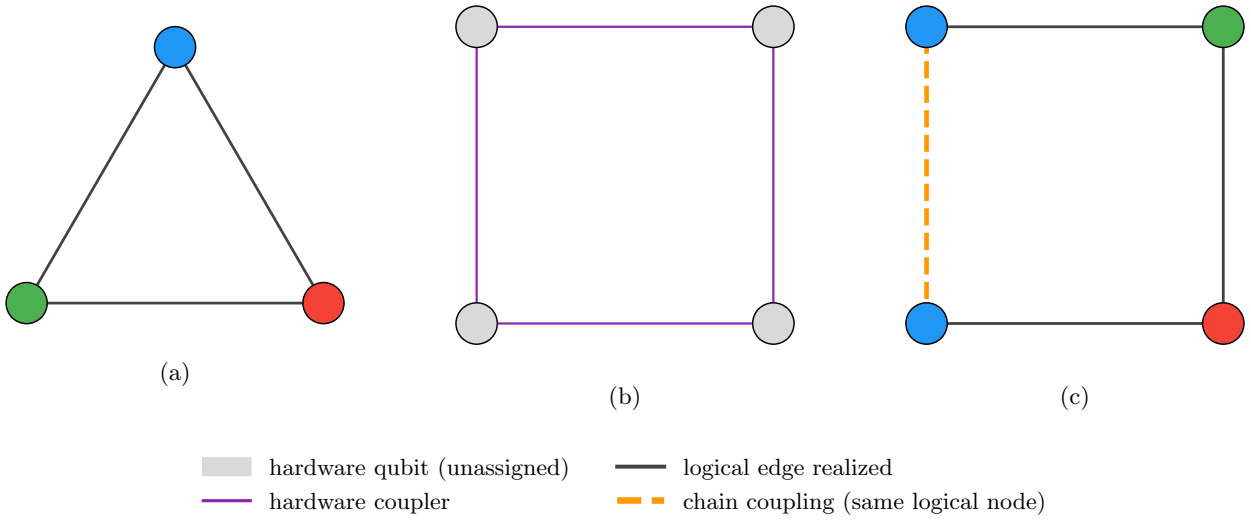
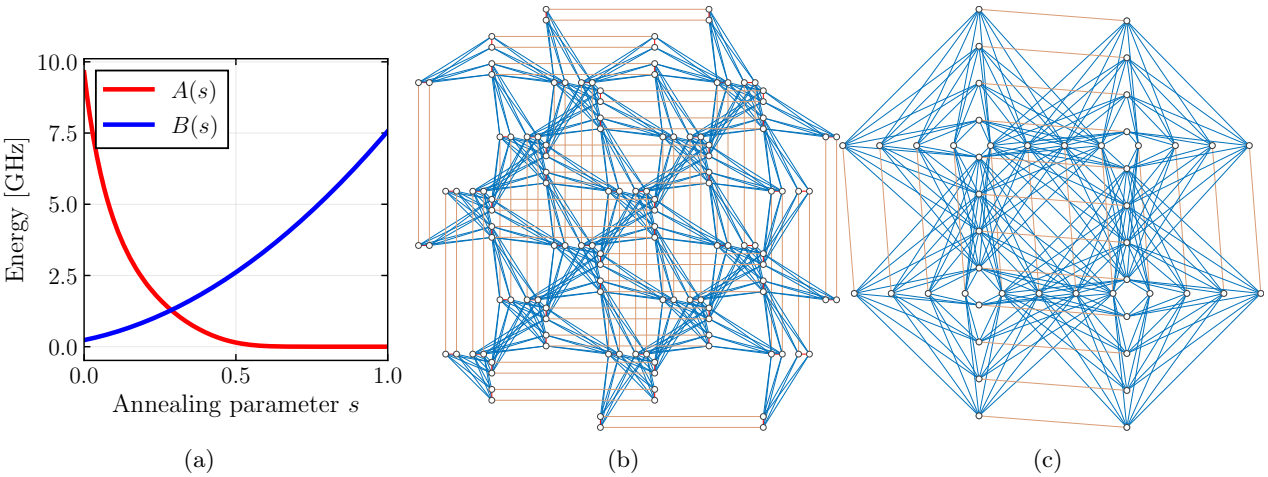


Figure 2: Minor embedding of a triangle onto a sparser hardware graph. **(a)** Logical problem graph: a triangle requires a coupler between every pair of nodes. **(b)** Target hardware graph: a 4-cycle, whose qubits admit no triangle directly. **(c)** A valid embedding: one logical node is realized as a chain of two physical qubits (joined by a strong ferromagnetic coupling, dashed), so that the three logical edges are each realized by a single physical coupler.

- **Internal Couplers** connecting qubits of opposite orientation
- **External Couplers** connecting qubits aligned in parallel
- **Odd Couplers** which connect similarly aligned pairs of qubits

See Figure 3 for visualization of different QPU topologies.



(a) (b) (c)

Figure 3: (a) Time-dependent energy scales executed by the D-Wave quantum annealer. The coefficient $A(s)$ (red) multiplies the transverse-field term that enables quantum tunnelling and therefore falls rapidly as the run progresses. The coefficient $B(s)$ (blue) multiplies the problem Hamiltonian and rises smoothly, so that the device moves from a tunnelling-dominated regime at $s = 0$ to a problem-dominated regime at $s = 1$. (b) Native connectivity map for the Pegasus architecture (D-Wave Advantage). Circles mark individual superconducting qubits; blue lines are internal couplers and orange lines mark external couplers (c) Connectivity map for the newer Zephyr architecture (D-Wave Advantage2), which increases the number of couplers per qubit and introduces longer-range links.

2.3.2 Simulated Bifurcation

Simulated Bifurcation (SB) is a heuristic algorithm that numerically simulates adiabatic and ergodic (chaotic) evolutions of classical nonlinear Hamiltonian systems to solve combinatorial optimization problems. Inspired by quantum adiabatic optimization using networks of Kerr-nonlinear parametric oscillators (KPO), the algorithm encodes the ground state of an Ising Hamiltonian into the stable states of a network of classical oscillators.

Each Ising spin is represented by continuous dynamical variables, position x_i and momentum y_i, which serve as canonical conjugate variables for each oscillator. The system evolves according to coupled differential equations derived from a network of Duffing oscillators. These equations include a nonlinear x^4 term and a time-dependent pumping parameter $p(t)$. As $p(t)$ is gradually increased, the system undergoes an adiabatic bifurcation, in which a single stable equilibrium at the origin splits into two stable branches. This mechanism drives the trajectories toward configurations corresponding to low-energy states of the Ising Hamiltonian. Because all variables are updated simultaneously at each time step, the algorithm is highly parallelizable and well suited for implementation on FPGAs and GPUs.

Langevin Simulated Bifurcation Langevin Simulated Bifurcation (LSB) is a stochastic adaptation of SB designed to function as a Boltzmann sampler for training energy-based models such as Semi-Restricted Boltzmann Machine (SRBM), which allow connections between visible units. While SB is a deterministic optimization method, LSB introduces stochasticity to approximate a Boltzmann distribution $B_{\beta_{\text{eff}}}$ with an unknown effective inverse temperature β_{eff}. To handle the unknown β_{eff} during learning, LSB is often combined with Conditional Expectation Matching (CEM). A detailed analysis of different temperature parameters is provided in subsection 4.3.

Conditional Expectation Matching CEM estimates the sampler's effective inverse temperature β_{eff} by matching the sampler's observed hidden-unit statistics, at a fixed visible configuration, against their closed-form conditional expectation [6]; the full derivation, including the pooled variant used in this work, is given in subsection 4.3. CEM can also be used with other temperature-dependent solvers, e.g. the D-Wave Quantum Annealer.

2.4 Metropolis-Hastings inspired methods

We use several Markov chain Monte Carlo (MCMC) methods to generate samples from probability distribution from which direct sampling would be difficult. All of these methods can be traced back to the Metropolis-Hastings algorithm, which we explain below.

2.4.1 Metropolis-Hastings Algorithm

The Metropolis-Hastings Algorithm is a MCMC method used to obtain samples from a (multivariate) probability distribution $P(x)$ from which sampling is difficult. The goal of the algorithm is to define a Markov chain whose stationary distribution is $P(x)$. Suppose we can evaluate an unnormalized target density $f(x)$, where $f(x) \propto P(x)$, and the normalization constant is unknown. In the **proposal** step, a *candidate* x^* from a proposal distribution $q(x^*|x_n)$ is generated, x_n being the current state of the Markov chain. In the **accept-reject** step, the acceptance probability $A(x_n \to x^*)$ is calculated[7]:

$$A(x_n \to x^*) = \min\left(1, \frac{f(x^*)}{f(x_n)} \times \frac{q(x_n|x^*)}{q(x^*|x_n)}\right). \tag{8}$$

For the evaluation of $\frac{f(x^*)}{f(x_n)}$, knowledge of the normalization constant is not necessary. The candidate x^* is accepted with probability $A(x_n \to x^*)$.

2.4.2 Simulated Annealing

simulated annealing (SA) is the classical analogue and predecessor of quantum annealing. It works by exploring the neighborhood of a given configuration, accepting the proposed solution with a temperature dependent probability, or if the new configuration has a lower energy than the current one.

2.4.3 Gibbs sampling

Gibbs sampling can be considered a special case of the Metropolis-Hastings algorithm. It is used when sampling directly from a joint distribution is difficult, but sampling from the full conditional distributions is tractable. The algorithm constructs a Markov chain by iteratively sampling each variable (or block of variables) from its distribution conditioned on the current values of all other variables. This procedure generates a chain whose stationary distribution is the target joint distribution.

3 Variational Algorithms

Variational algorithms are a class of hybrid quantum-classical optimization methods designed to find the ground state of a given Hamiltonian. Unlike the Variational Quantum Eigensolver (VQE), which prepares and measures a parametrised circuit on a gate-based quantum computer, this work uses quantum-assisted Variational Monte Carlo: a classical neural-network ansatz is optimised via Stochastic Reconfiguration, and a quantum device is used only to generate samples of that ansatz. Variational Monte Carlo consists of three parts: the *cost function* to be minimized, the *ansatz*, which in our case is a Boltzmann machine, and an *expectation value estimation*. Additionally, a classical optimizer is needed to improve the ansatz at each iteration. Depending on how the ansatz is realised on quantum hardware, two approaches for estimating the expectation value are possible:

- On gate-based quantum computers, measuring can be used to gather expectation values of the modified ansatz and improve it further
- In adiabatic quantum computing, the quantum device is used to generate representative samples of the modified ansatz. These samples are then used to approximate the expectation value.

As we are interested in comparing current quantum annealing devices with their classical counterparts, we focus solely on the second case. The goal is to approximate the ground state, that is, the eigenstate corresponding to the lowest eigenvalue of a Hamiltonian H. In the adiabatic case, the ansatz is defined via an energy function over visible and hidden units,

$$E_{\boldsymbol{\theta}}(\mathbf{v}, \mathbf{h}) = \mathbf{a} \cdot \mathbf{v} - \mathbf{b} \cdot \mathbf{h} - \mathbf{h} \cdot \mathbf{W}^\top \cdot \mathbf{v}, \tag{9}$$

with weights $\boldsymbol{\theta} = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$ respectively applied to the vector of visible units $\mathbf{v} = (v_1, \ldots, v_N) \in \{-1, +1\}^N$ and hidden units $\mathbf{h} = (h_1, \ldots, h_M) \in \{-1, +1\}^M$ – both Ising-valued rather than the more common $\{0, 1\}$ Bernoulli convention, which is what gives Equation 12's $2\cosh(\cdot)$ factor and the conditional expectation $\langle h_j \rangle = \tanh(\beta\Theta_j)$ used in subsection 4.3 their specific form. Throughout, $\mathbf{W} \in \mathbb{R}^{N \times M}$ is indexed visible-by-hidden, W_{ij} with i the visible and j the hidden index, matching its representation in the implementation. The joint quantum state is

$$|\Psi\rangle = \sum_{\mathbf{v}} \Psi(\mathbf{v}) \, |\mathbf{v}\rangle \,, \qquad \Psi(\mathbf{v}) = \sqrt{\sum_{\mathbf{h}} e^{-E_{\boldsymbol{\theta}}(\mathbf{v}, \mathbf{h})}}, \tag{10}$$

so that $|\Psi(\mathbf{v})|^2 = \sum_{\mathbf{h}} e^{-E_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h})}$ is, up to normalization, the marginal over hidden units of the joint Boltzmann distribution $p_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h}) = Z^{-1} e^{-E_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h})}$, $Z = \sum_{\mathbf{v},\mathbf{h}} e^{-E_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h})}$, i.e. $|\Psi(\mathbf{v})|^2 \propto \sum_{\mathbf{h}} p_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h})$. The corresponding Born probability distribution is

$$\rho(\mathbf{v}) = \frac{|\Psi(\mathbf{v})|^2}{\sum_{\mathbf{v}'} |\Psi(\mathbf{v}')|^2}, \tag{11}$$

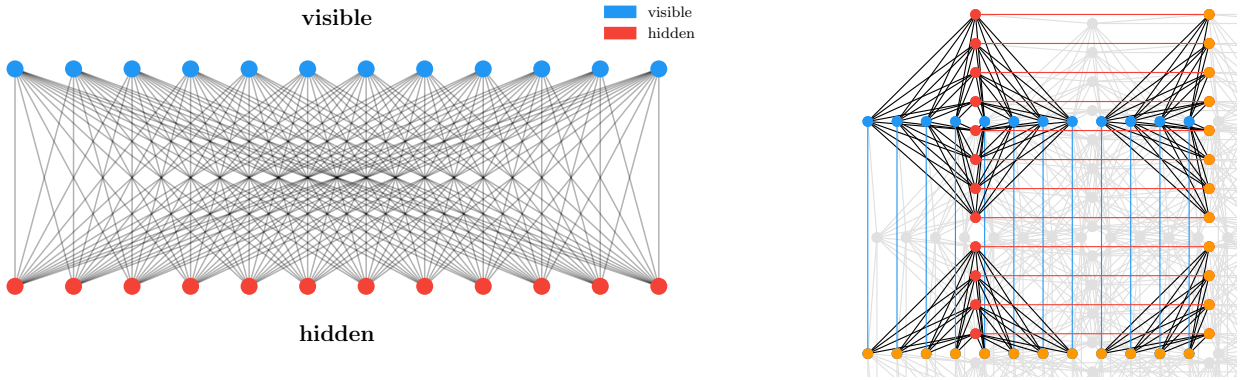
which is however intractable to normalize for large Hilbert spaces. Tracing out the hidden variables in Equation 10, the ansatz becomes:

$$\Psi(\mathbf{v}) = e^{-\mathbf{a}\cdot\mathbf{v}/2} \prod_{j=1}^{N_h} \left[2\cosh\left(b_j + \sum_i W_{ij} v_i \right) \right]^{1/2}. \tag{12}$$

Two sampling regimes are used in this work. Classical Metropolis-Hastings sampling operates directly on the closed-form marginal $\Psi(\mathbf{v})$ of Equation 12: only the visible units are instantiated, and $\mathbf{h}$ never appears as an explicit random variable. Gibbs sampling on the D-Wave annealer instead targets the joint distribution $p_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h})$: both $\mathbf{v}$ and $\mathbf{h}$ are encoded as physical qubits (Figure 4), the annealer returns samples of the pair $(\mathbf{v},\mathbf{h})$, and $\mathbf{h}$ is subsequently discarded (marginalized) to obtain samples of $\mathbf{v}$ distributed as $\rho(\mathbf{v})$. Since D-Wave qubit biases and coupler strengths are restricted to a fixed hardware range, $\mathbf{a}, \mathbf{b}, \mathbf{W}$ are rescaled by a common factor β_x before being programmed onto the device; under `auto_scale=False` this rescaling is the origin of the effective inverse temperature β_{eff} discussed in subsection 4.3, though subsection 4.3 shows this mechanism does not hold once `auto_scale` is enabled.

3.1 Boltzmann machine

A restricted Boltzmann machine (RBM) is an artificial neural network consisting of visible and hidden units that form a bipartite graph. That is, no intra-layer connections are allowed. During training, the RBM learns to approximate a probability distribution. The main idea is that expectation values with respect to the quantum state can be estimated using samples drawn from the probability distribution $|\Psi(\mathbf{v})|^2$. Since the wavefunction ansatz is represented by a Restricted Boltzmann Machine, which corresponds to a classical stochastic Ising model, configurations $\mathbf{v}$ can be sampled approximately using a D-Wave quantum annealer.



(a) (b)

Figure 4: (a) RBM with 24 visible and 24 hidden units. (b) Embedding of the corresponding bipartite graph onto the Zephyr hardware topology. Blue nodes are visible units, red nodes are hidden units, orange nodes are auxiliary chain qubits. Each physical node is connected to one auxiliary qubit.

4 Experimental Analysis

We conduct experiments analyzing sampler quality and physical properties of samplers and models.

4.1 Sampling Quality

While some of the samplers are guaranteed to sample from the Boltzmann distribution, others are not. In this section, we analyze the quality of samplers with regard to various parameters. One measure for sampling quality is the *total variation distance* D_{TV}. It is defined as

$$D_{\text{TV}}(\mu,\nu) = \frac{1}{2} \sum_{\boldsymbol{\sigma}\in\{-1,+1\}^N} |\mu(\boldsymbol{\sigma}) - \nu(\boldsymbol{\sigma})|, \tag{13}$$

where μ is the histogram returned from the sampler and $\nu(\boldsymbol{\sigma}) \propto |\Psi(\boldsymbol{\sigma})|^2$ is the RBM distribution. As every sampler has a range of parameters, we are interested in how well the sampling performs with regards to the different tuning parameters. SA, Gibbs, and Metropolis can be tuned by the number of *sweeps*, that is, the number of single-spin-flip proposals, whereas LSB is tunable by the number of *steps* which updates all spins simultaneously.

We use the following workflow to determine D_{TV}: First, we use a SA pretrained RBM and compute its exact distribution $|\Psi|^2$ numerically by enumerating all possible spin configurations. We then collect samples from different samplers and compare the resulting empirical histogram to the exact distribution.

The results are presented in Figure 6. As a sanity check, it can be seen that Metropolis and Gibbs sampling perform theoretically near-optimal, as is expected. We observe that LSB significantly underperforms the other solvers for this small-scale example. The performance of both LSB and SA improves as the transverse field bias h increases. In Figure 6**(b)** the exact histogram of configurations $|\Psi(\mathbf{v})|^2$ is plotted. For $h = 0.5$ the distribution is peaked around a handful of highly probable sample configurations, whereas the distribution becomes more uniform for stronger values of h. This explains the performance increase for LSB and SA samplers.

The peaked distribution originates from the fact that the spins in the TFIM description given by Equation 2 are aligned for low h values (*ferromagnetic* phase). For $h \to \infty$ the transverse field dominates and the ground state becomes a product of single-spin σ_x eigenstates. Measured in the z-basis, this leads to an exactly uniform distribution over spin configurations for any N. Figure 5 illustrates this directly: sampled spin configurations and correlation functions at three values of h, spanning the ferromagnetic and paramagnetic regimes.

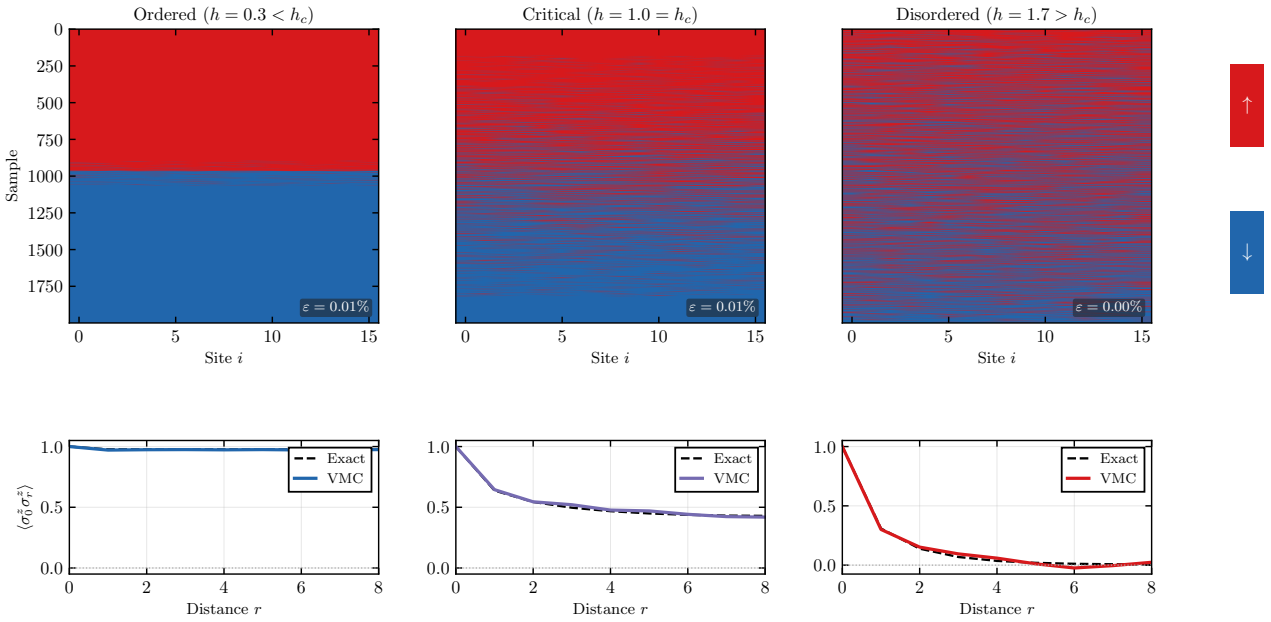
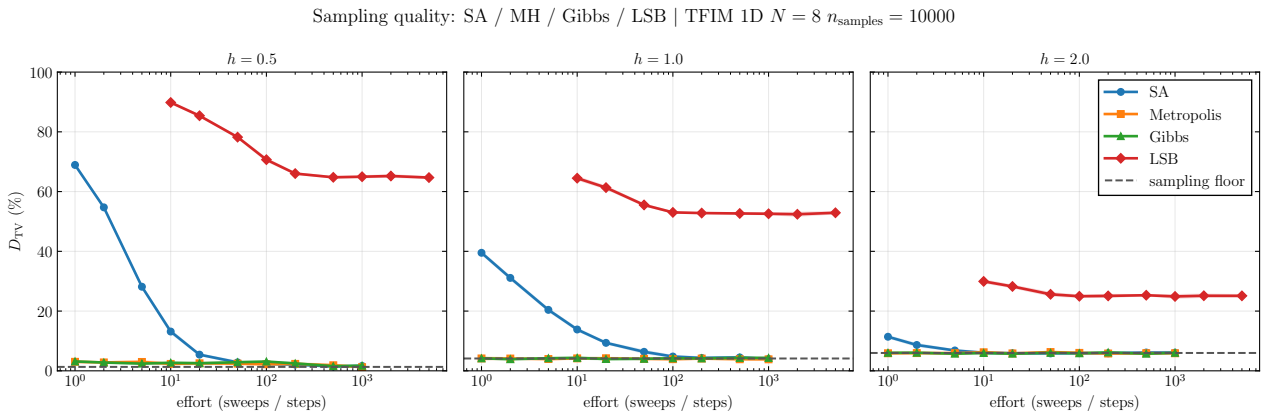


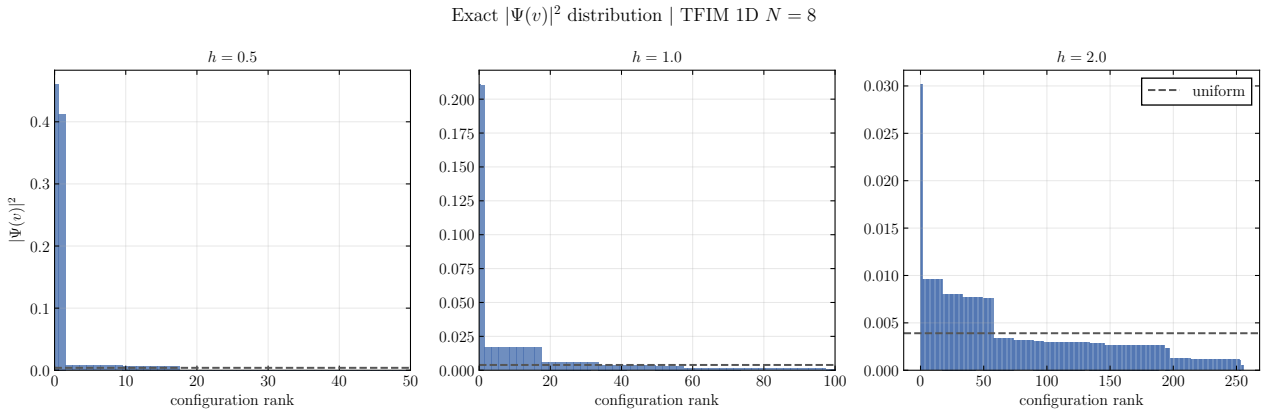
Figure 5: Spin configurations sampled from a trained RBM-VMC model for the one-dimensional Transverse-Field Ising Model (TFIM) at three values of the transverse field h. **Top row:** each column shows 2 000 spin configurations $\boldsymbol{\sigma} \in \{-1, +1\}^N$ sampled from the learned wave function $|\Psi_{\boldsymbol{\theta}}|^2$, sorted by total magnetisation. **Bottom row:** spin-spin correlation $C(r) = \langle \sigma_0^z \sigma_r^z \rangle$ as a function of distance r, computed from VMC samples (solid line) and exact diagonalisation (dashed line). The relative energy error $\varepsilon = |E_{\mathrm{VMC}} - E_{\mathrm{exact}}|/|E_{\mathrm{exact}}|$ is shown in each panel. The model is a fully-connected RBM ($N = N_h = 16$) trained for 300 iterations of Stochastic Reconfiguration with 500 Metropolis-Hastings samples per step.

4.2 Time-to-ϵ across solvers

We next compare solvers by time to ϵ (TTE), the wall-clock time until training reaches a *validated* convergence, defined as an energy error per spin below $\epsilon = 0.1$. A practitioner deploying one of these solvers on a new problem has no access to the exact ground-state energy, so a useful stopping rule must be oracle-free: we monitor the coefficient of variation $\mathrm{CV} = \mathrm{std}(E_{\mathrm{loc}})/|\mathrm{mean}(E_{\mathrm{loc}})|$ of the local energy and mark a run as self-converged once CV has stayed below 0.05 for 10 consecutive iterations. A run only counts as validated, however, if the energy averaged over that self-detected plateau is, in fact, within ϵ of the true ground-state energy; a run that self-detects convergence at the wrong plateau – which we find does happen for some D-Wave cells at small N – is censored rather than counted, so that TTE reports time to a genuinely good answer rather than merely time to training appearing to stop.



(a) Total variation distance as a function of sampling effort.



(b) Exact histogram of configurations.

Figure 6: Sampling quality for different samplers for a trained RBM approximating the ground state of a TFIM with 8 spins, corresponding to a RBM with 8 visible and 8 hidden units. The sampling floor is the best possible total variation distance reachable with the given number of samples. Effort is measured in sweeps for SA, metropolis and Gibbs, and in steps for LSB.

Every series shares the same training protocol: learning rate 0.08, regularization 0.05, 200 samples per iteration, a 100-iteration budget, and 20 seeds per cell. This is a matched-hyperparameter comparison, not a matched-tuning-effort one, and should be read as such. For the D-Wave runs, the reported time is the QPU's own `qpu_access_time` (on-chip programming, anneal, and readout), with network latency and queueing excluded and embedding treated as a one-time cost paid outside the training loop. The comparison should also not be over-read as temperature-comparable: the effective inverse temperature β_{eff} of the QPU runs is not independently calibrated here (see subsection 4.3 for how β_{eff} is estimated elsewhere in this work).

Figure 7 plots the median TTE (with interquartile range) against system size $N \in \{8, 12, 16, 24, 32, 64\}$, split across three panels sharing a y-axis so that absolute scale stays directly comparable: **(a)** classical MCMC samplers with an asymptotic-correctness guarantee (Metropolis, Gibbs); **(b)** classical, physics-inspired heuristics with no such guarantee (LSB with CEM, SA on VeloxQ [8] left untuned, and FPGA); and **(c)** the Pegasus and Zephyr QPUs, each with and without CEM. Splitting the nine series this way, rather than plotting them on one shared axis, is what keeps the classical-versus-quantum comparison in panel (c) legible. We omit $N = 128$ from the figure: at that size the fixed 100-iteration training budget does not bring any solver within ϵ of the exact ground-state energy, which we confirmed directly rather than relying on the self-detector, so every series would censor there for the same training-budget reason rather than any real difference in sampler speed. Plotting $N = 128$ would therefore invite a solver-versus-solver reading that the data cannot support, and the omission should not be mistaken for a QPU coverage gap – Pegasus and Zephyr are both represented at every other size, $N \in \{8, 16, 32, 64\}$.

Across the plotted range, panel (b)'s FPGA is consistently fastest, followed by SA, then panel (c)'s two QPUs, with panel (a)'s Metropolis and Gibbs and panel (b)'s LSB the slowest; this ordering is a trend rather than a strict rule, since SA and FPGA become heavily censored at $N = 64$ (only 10/20 and 6/20 seeds validate there, respectively) and both QPUs overtake them at that size. Most cells validate on most or all of their 20 seeds, but two are fully censored, both in panel (c): the Zephyr QPU without CEM validates 0/20 seeds at $N = 8$, and the Pegasus QPU without CEM validates 0/20 seeds at $N = 16$. Both are plotted as hollow markers at the run's actual elapsed time at the 100-iteration budget, not an extrapolated value. The CEM-corrected Pegasus and Zephyr series validate far more consistently at these same sizes, consistent with CEM correcting the effective-temperature miscalibration discussed in subsection 4.3.

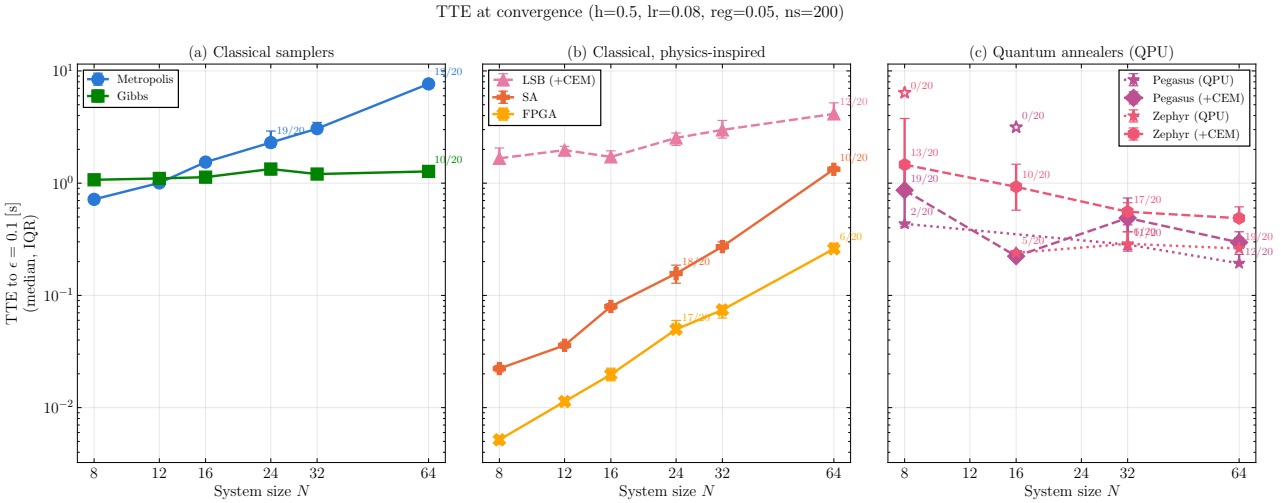


Figure 7: Median time-to-ϵ (interquartile range) for $\epsilon = 0.1$ energy error per spin, against system size N, under the shared protocol described in the text (lr= 0.08, reg= 0.05, 200 samples/iteration, 100-iteration budget, 20 seeds/cell). Solvers are grouped into three panels on a shared y-axis: **(a)** classical samplers, **(b)** classical physics-inspired heuristics, **(c)** quantum annealers (QPU). Filled markers give the median over seeds whose self-detected convergence is validated against the true ground-state energy; a marker is annotated with the validated-over-total seed count whenever that fraction is below 20/20. Hollow markers mark a cell where no seed validates and are placed at the run's actual elapsed time at the iteration budget rather than an extrapolated value. Pegasus and Zephyr times in panel (c) are D-Wave's `qpu_access_time` only, excluding network/queueing time and the one-time embedding cost. $N = 128$ is omitted for every solver because the training budget, not any solver's sampling speed, is the bottleneck at that size.

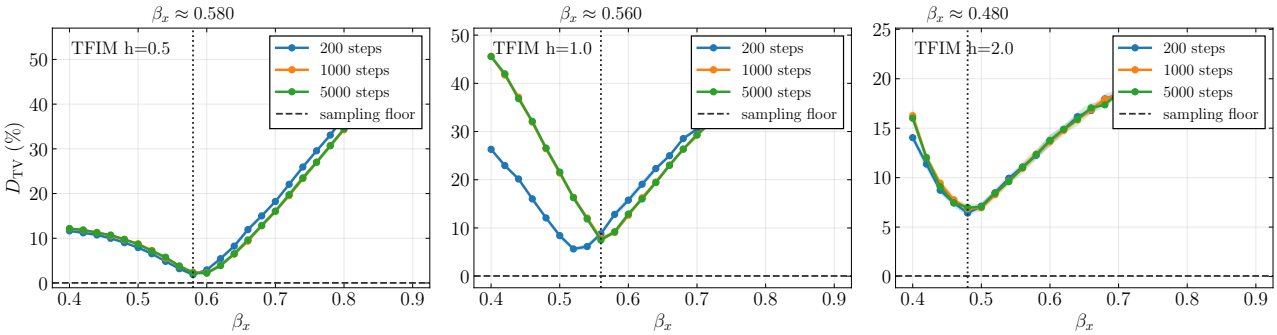
4.3 Sampling temperature & CEM

In this section, we evaluate the influence of temperature on different models and solvers. We focus on two quantities: the rescaling factor β_x and the effective temperature β_{eff}. Sampling from an ansatz, all weights in the RBM are rescaled by β_x, that is, the sampler is programmed with $E_{\text{in}} = \alpha E_{\boldsymbol{\theta}}$, $\alpha = 1/\beta_x$. If the device samples at hardware inverse temperature β_{hw}, it draws from $p(\mathbf{v}, \mathbf{h}) \propto e^{-\beta_{\text{hw}} E_{\text{in}}(\mathbf{v},\mathbf{h})} = e^{-(\alpha\beta_{\text{hw}}) E_{\boldsymbol{\theta}}(\mathbf{v},\mathbf{h})}$, i.e. relative to the unscaled model $\beta_{\text{eff}} = \alpha\beta_{\text{hw}}$. For an ideal Gibbs sampler held at $\beta_{\text{hw}} = 1$, the following equation holds:

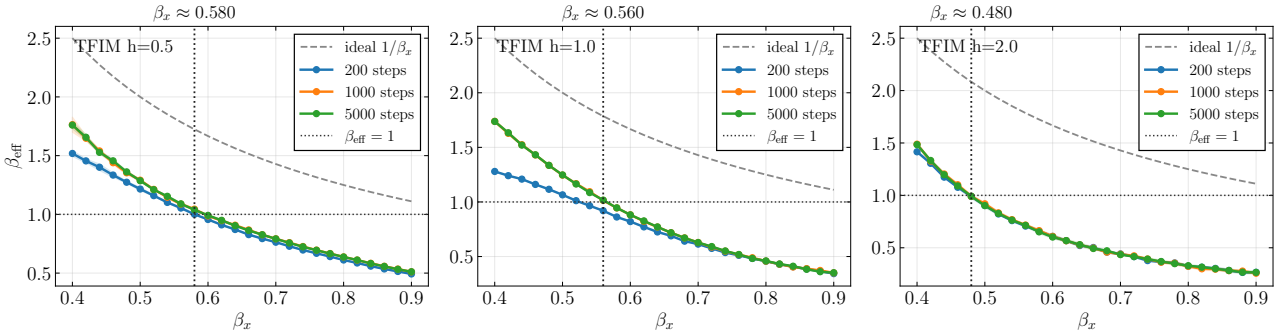
$$\beta_{\text{eff}} = \frac{1}{\beta_x}. \tag{14}$$

4.3.1 Langevin Simulated Bifurcation

As LSB is a temperature dependent solver and not guaranteed to sample from the Boltzmann distribution, it can be used to illustrate the influence of the input temperature β_x on the solution quality. Contrary to the quantum annealer, sampling using LSB does not involve additional costs. To evaluate the impact of the scaling parameter β_x, we calculate the total variation distance D_{TV} between the samples and the precise Boltzmann distribution by numerical enumeration. Figure 8 shows that the value of β_x minimizing D_{TV} shifts modestly with the transverse field h: the optimum sits at $\beta_x \approx 0.580$, 0.560, 0.480 for $h = 0.5, 1.0, 2.0$ respectively – a spread of about 20% over a 4× change in h, with D_{TV} rising clearly on both sides of each optimum. D_{TV} is minimal near the point where the effective temperature β_{eff} is 1.



(a) Total variation distance D_{TV} between the LSB empirical histogram and the exact distribution $|\Psi(v)|^2$. The horizontal dashed line marks the sampling floor.



(b) Effective inverse temperature β_{eff} of the LSB samples, estimated via KL-argmin ($\beta_{\text{eff}} = \arg\min_\beta D_{\text{KL}}(p_S \parallel p_\beta)$, subsubsection 4.3.2). The dashed curve shows the ideal $1/\beta_x$ prediction for a perfect Gibbs sampler, the horizontal dotted line marks the VMC target $\beta_{\text{eff}} = 1$.

Figure 8: Influence of the LSB energy-scale parameter β_x on sampling quality for a trained TFIM RBM with $N = 8$ visible spins. The vertical dotted line in each panel marks the β_x that minimises D_{TV}.

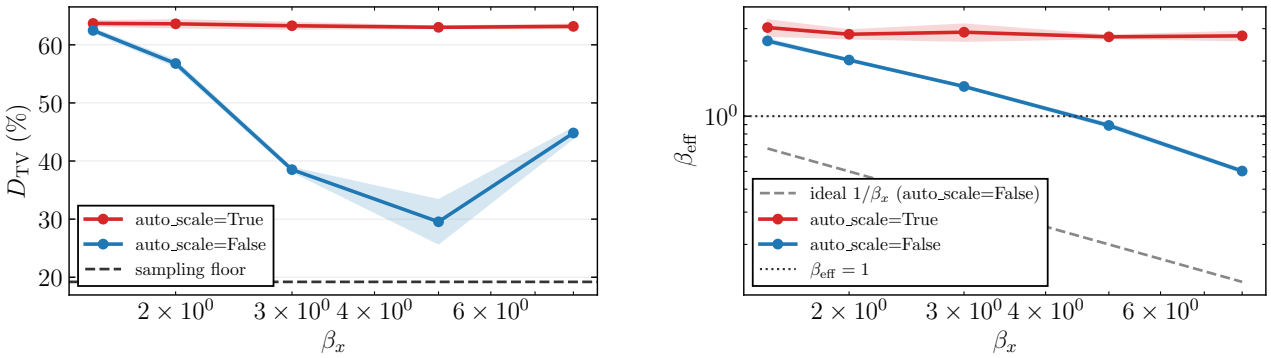
4.3.2 Auto-Scaling on the D-Wave QPU

Submitting a problem to a D-Wave QPU adds a rescaling step, `auto_scale`, enabled by default (subsection 5.4): SAPI renormalizes the submitted h and J to make full use of the solver's allowed coefficient range, based only on their *relative* magnitudes. Since β_x rescales h and J *uniformly*, `auto_scale` absorbs that rescaling before the anneal even runs – so β_{eff} should be essentially independent of β_x whenever `auto_scale=True`, unlike the $\beta_{\text{eff}} = 1/\beta_x$ behaviour observed for LSB above.

We verify this on the `Advantage_system6` (Pegasus) chip, using a chain-free RBM embedded directly onto a native Pegasus biclique so chain breakage cannot confound the result. Figure 9 sweeps β_x under both settings: with `auto_scale=True`, β_{eff} stays flat at ≈ 2.7–3.0 across a $5\times$ range of β_x; with `auto_scale=False` it tracks $\approx 4.1/\beta_x$, recovering the ideal $1/\beta_x$ line up to a constant prefactor (≈ 4.1 in this configuration). Every other D-Wave result in this work uses the default `auto_scale=True`, so wherever β_x was varied for QPU-sampled runs, it had no effect on the actual physical sampling temperature.

The sweep reuses the sampler already used throughout this work (full script: `scripts/dtv_autoscale.py`):

```
cfg = {"beta_x": beta_x, "auto_scale": auto_scale,
       "annealing_time": annealing_time}
v = dwave_sampler.sample(rbm, num_reads, config=cfg)
p_emp = empirical_dist_jax(v, N)
beta_eff = estimate_beta_eff(v, p_emp)  # argmin_beta KL(p_emp || p_beta)
```



(a) Total variation distance D_{TV} between the QPU samples and $|\Psi(v)|^2$.

(b) Effective inverse temperature β_{eff}, with the ideal $1/\beta_x$ line and the $\beta_{\text{eff}} = 1$ target.

Figure 9: Impact of `auto_scale` on β_x for a chain-free Pegasus-embedded TFIM RBM ($N = 8$, $h = 1$). `auto_scale=True` (red) makes β_{eff} independent of β_x; `auto_scale=False` (blue) restores the intended $1/\beta_x$ control.

4.3.3 CEM

We have seen that correctly choosing the sampling temperature is crucial in obtaining meaningful distributions. CEM's original formulation [6] exploits that the hidden units of a Boltzmann machine are conditionally independent given the visible units: for a visible configuration fixed to a condition vector $\mathbf{v} = \mathbf{r}$, the conditional expectation $\langle h_j \rangle = \tanh(\beta\Theta_j)$, $\Theta_j = b_j + \sum_i r_i W_{ij}$, is known in closed form for any β. To estimate the unknown β_{eff}, the sampler is queried repeatedly at the fixed condition $\mathbf{r}$ to obtain an empirical conditional average $\langle h_j \rangle_{\mathbf{r},C}$, which is then matched against $\tanh(\beta\Theta_j)$ by minimizing the squared residual over β.

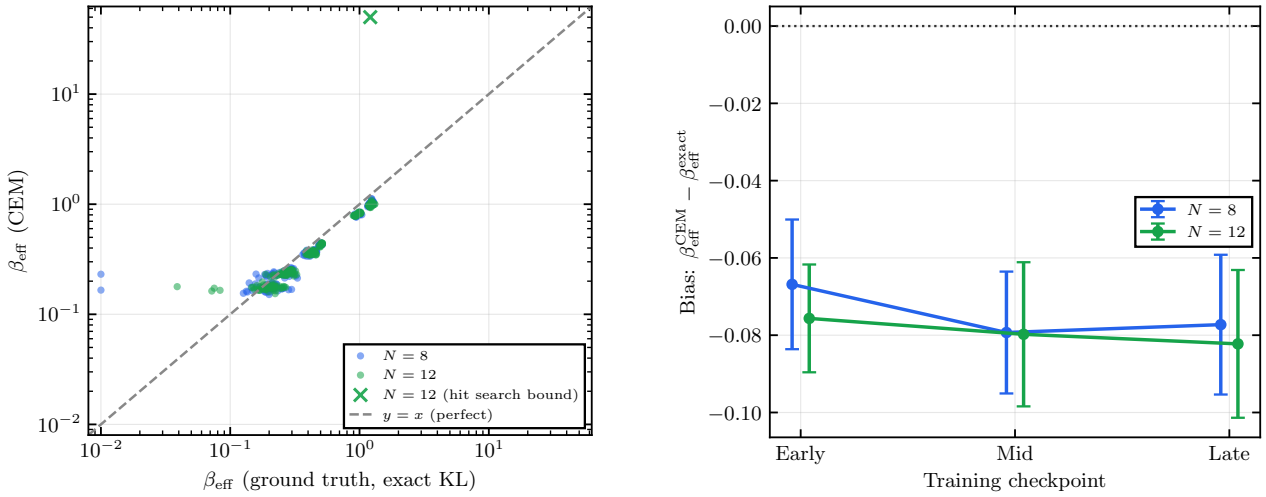
In this work we use a pooled variant of this matching step that avoids the dedicated conditional-sampling calls the original procedure requires. Rather than fixing $\mathbf{v}$ and drawing samples at that one condition, we reuse the ordinary joint samples $(\mathbf{v}_i, \mathbf{h}_i)_{i=1}^{N_s}$ the sampler already produces at each training iteration – each with its own, generally distinct, visible configuration $\mathbf{v}_i$ – and fit a single β_{eff} across the whole batch,

$$\beta_{\text{eff}} = \arg\min_{\beta} F(\beta), \quad F(\beta) = \sum_{i=1}^{N_s} \sum_{j=1}^{N_h} \left(h_{ij} - \tanh(\beta \cdot \Theta_j(\mathbf{v}_i))\right)^2, \tag{15}$$

where i is now repurposed as the sample index (rather than the visible-unit index it denoted above), h_{ij} is the observed hidden unit j in sample i, and $\Theta_j(\mathbf{v}_i) = b_j + \sum_l v_{i,l} W_{lj}$ with l the visible-unit index. This adds no marginal sampler cost while pooling $N_s \times N_h$ residuals per estimate instead of N_h residuals at a single condition.

The matching step is visualized in Figure 11, where the left subfigure shows multiple candidate functions $\tanh(\beta\Theta)$ dependent on β that can be used to match the returned samples. The right subfigure describes the corresponding minimization, in which the squared loss function $F(\beta)$ is minimized over β (in practice using scipy's `minimize_scalar` function). We validate this matching procedure against an independent, exact ground-truth estimate – obtained by minimising the KL divergence between the empirical visible-marginal distribution and the true visible marginal of the β-rescaled joint distribution, $p_\beta(\mathbf{v}) \propto e^{-\beta\, \mathbf{a}\cdot\mathbf{v}} \prod_j 2\cosh(\beta\Theta_j)$, via exact enumeration – across $N \in \{8, 12\}$, $h \in \{0.5, 1.0, 1.5, 2.0\}$, and $\beta_x \in \{0.5, 1.0, 1.5, 2.0\}$, with 5 seeds per condition and three checkpoints during training (see Figure 10). CEM shows a small, consistently negative bias at every

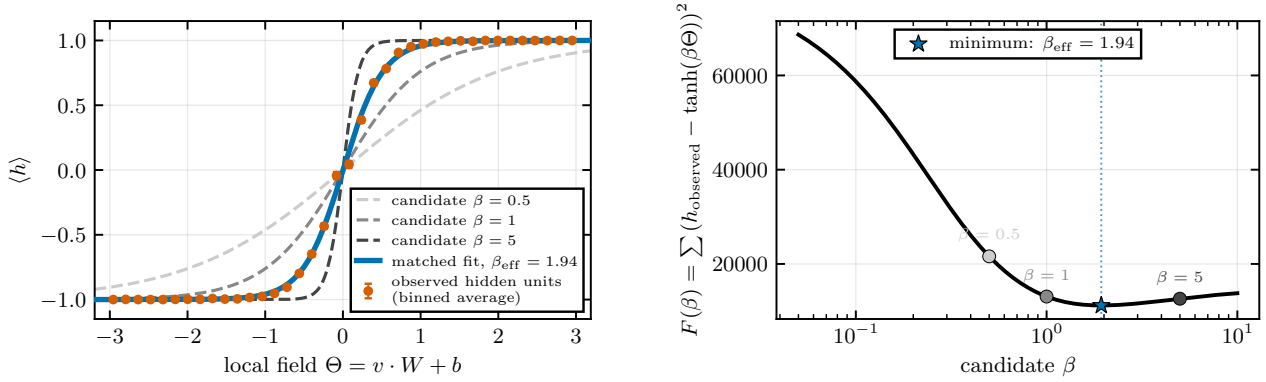
checkpoint, of order -0.07 to -0.08 when averaged over β_x and h, roughly flat across training rather than shrinking; excluding one draw where the fit saturated its optimizer's search bound (marked $\times$ in Figure 10(a), and excluded from both the root mean squared error (RMSE) below and panel (b)'s bias averages), its RMSE against the exact estimate is 0.107 ($N = 8$) and 0.112 ($N = 12$). Conditioned on β_x instead, the bias is largest at low β_x ($\beta_x = 0.5$, mean bias ≈ -0.19) and shrinks as β_x grows (≈ -0.02 at $\beta_x = 2.0$).



(a) CEM vs. exact ground truth.

(b) Bias across training.

Figure 10: Validation of the CEM β_{eff} estimator against an independent exact estimate (KL-argmin against the true visible marginal of the β-rescaled joint Boltzmann distribution), pooled over $N \in \{8, 12\}$, $h \in \{0.5, 1.0, 1.5, 2.0\}$, $\beta_x \in \{0.5, 1.0, 1.5, 2.0\}$, and 5 seeds per condition (480 draws total). (a) CEM's estimate vs. the exact ground truth; the dashed line marks perfect agreement. One draw (marked $\times$) saturated CEM's optimizer search bound and is excluded from the RMSE and from panel (b). (b) Bias of the CEM estimate relative to ground truth, averaged over h and β_x, at three checkpoints during training (10%/50%/100% of iterations). Error bars show 95% confidence intervals over seeds.



(a) Candidate fits vs. observed hidden units.

(b) Matching objective $F(\beta)$.

Figure 11: The CEM matching step for $N = 16$ spins with $h = 1.0$. (a) Several candidate $\tanh(\beta\Theta)$ curves against the empirically observed hidden-unit samples. The curve at $\beta_{\text{eff}} = 1.94$ approximates the data most closely and represents the best estimate of the sampler's temperature. (b) The squared loss function to be minimized.

4.4 Limitations of variational methods

In this section we describe computational and theoretical limitations to using variational methods in a hybrid quantum-classical setup.

4.4.1 The Sign Problem

It is well known that capturing quantum dynamics with non-trivial sign structure using a real-valued variational ansatz is computationally hard (*sign problem*). This does not pose a problem for the TFIM models, as its ground state has only positive amplitudes. More complex models, such as some J_1-J_2 Heisenberg models, on the other hand cannot be simulated using our methods. The reason for this is that the factors $e^{-\mathbf{a}\cdot\mathbf{v}/2}$ and $\cosh(\cdot)$ of the ansatz defined in Equation 12 are strictly positive for real parameters and therefore the ansatz does not allow for the representation of negative amplitudes.

The Marshall sign rule For certain Heisenberg models with bipartite lattice structure, the Marshall sign rule can be used to remove the non-trivial sign structure of the ground state wavefunction by basis transformation. Considering the following unitary transformation on bipartite lattices $A \cup B$, written in the Pauli convention used throughout this work:

$$U = \prod_{i \in A} \sigma_i^z,$$

the ground state wavefunction can be written as

$$U|\psi_0\rangle = \sum_{\sigma} |c_\sigma|\,|\sigma\rangle,$$

with only positive amplitudes. This transformation is exact only for the unfrustrated case $J_2 = 0$, where the lattice is bipartite; for $J_2 > 0$ the next-nearest-neighbour bonds connect sites within the same sublattice, breaking bipartiteness, so applying the same sign pattern is a heuristic gauge choice rather than a proven sign-removal[9]. We measure the impact of the sign problem on a given probability distribution by the average sign[10], evaluated on the exact ground-state wavefunction Ψ_0 obtained by exact diagonalisation – not on the RBM ansatz Ψ_θ, which is strictly positive by construction and would give $\langle s \rangle \equiv 1$ trivially. It is given by:

$$\langle s \rangle = \frac{\sum_{\mathbf{v}} \Psi_0(\mathbf{v})}{\sum_{\mathbf{v}} |\Psi_0(\mathbf{v})|} \tag{16}$$

Interestingly, we empirically observe that even for frustrated Heisenberg models with $0 \leq \frac{J_2}{J_1} \leq 0.5$ the RBM with the Marshall sign correction can accurately approach the ground state energy, whereas without the correction the variational energy degrades significantly — consistent with [9], who found that the sign rule can empirically survive frustration over part of the parameter range even though the proof no longer strictly applies beyond $J_2 = 0$. Empirically, we observe that the median relative error over independent seeds rises sharply once J_2/J_1 exceeds 0.5 (Figure 12), in line with expectations that stronger frustration increasingly violates the positive-amplitude assumption underlying the ansatz. We did not test a phase-capable ansatz (e.g. a complex-valued RBM or a separate amplitude-and-phase network); such a comparison is left as future work.

4.5 Improving D-Wave Topology performance

A crucial factor in the success of an annealing process is the *problem embedding* on the chip. Here, we assess two aspects related to embedding: First, introducing sparsity to fit the RBM natively on the chip, and, second, embedding the problem multiple times to save computing time.

4.5.1 Sparse embedding

As described in subsubsection 2.3.1, problems implementable on a quantum annealer often need to be fit to the QPU in an additional embedding step. We are interested in whether a sparse, but native RBM, that is, one that fits natively on the QPU, performs reasonably. The sparse RBM is created according to the following algorithm: First, construct a bipartite graph by keeping only edges that correspond to internal couplers on the D-Wave QPU. One set of the graph corresponds to visible RBM units, the other one to hidden units. Then, greedily pick qubits for the visible units, by maximizing the overlap between neighbor sets of visible units. After all visible units are chosen, create a minimum set-cover by picking the fewest hidden qubits such that every visible qubit has at least one hidden neighbor. The remaining hidden units are filled by best connected qubits. See also Figure 13 for a visual overview of the algorithm. We also looked directly at the effect of sparsity on its own, keeping the number of hidden units equal to the number of visible units and only changing how many of the real couplers are removed from the connectivity mask. When removing couplers, we always removed one from the hidden unit with the highest remaining degree, so that no visible or hidden unit was ever left with zero connections. Figure 14 shows the resulting energy error as a function of sparsity, for three classical samplers (Metropolis-Hastings, simulated annealing, and persistent-chain Gibbs). In all three cases, error increases sharply as sparsity increases.

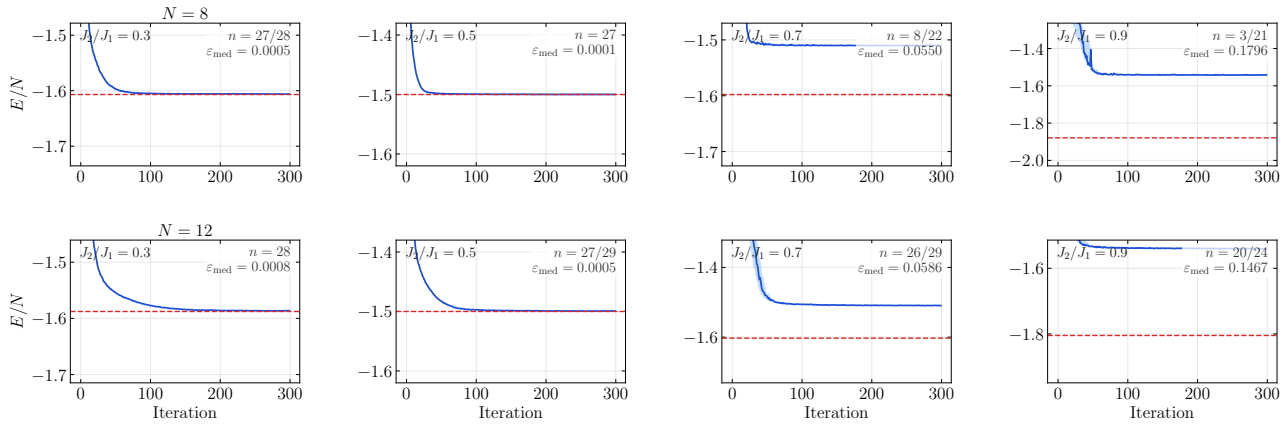


Figure 12: RBM convergence curves (E/N vs. SR iteration) for $N \in \{8, 12\}$ and $J_2/J_1 \in \{0.3, 0.5, 0.7, 0.9\}$ on the frustrated J_1–J_2 Heisenberg chain, using Gibbs sampling. The solid line is the median over n independent evaluation seeds (annotated per panel); the shaded band is the interquartile range. For each panel, the hyperparameter configuration was selected beforehand on a disjoint tuning set (Optuna trials, tuning seeds excluded from evaluation); ε_{med} denotes the median relative error of the tail-averaged energy. Where n is reported as a/b, b seeds were attempted and $b - a$ runs diverged (non-finite energy) under the selected configuration and are retained as failures rather than silently dropped; divergence is frequent in the strongly frustrated regime ($N = 8$, $J_2/J_1 \geq 0.7$). The red dashed line marks the exact ground-state energy E_{exact}/N. The median relative error rises sharply beyond $J_2/J_1 = 0.5$.

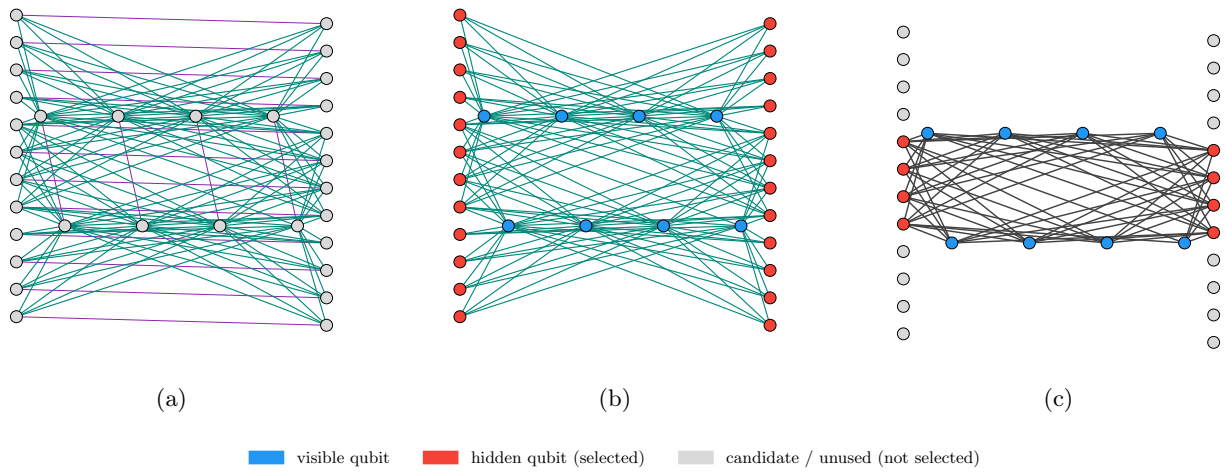
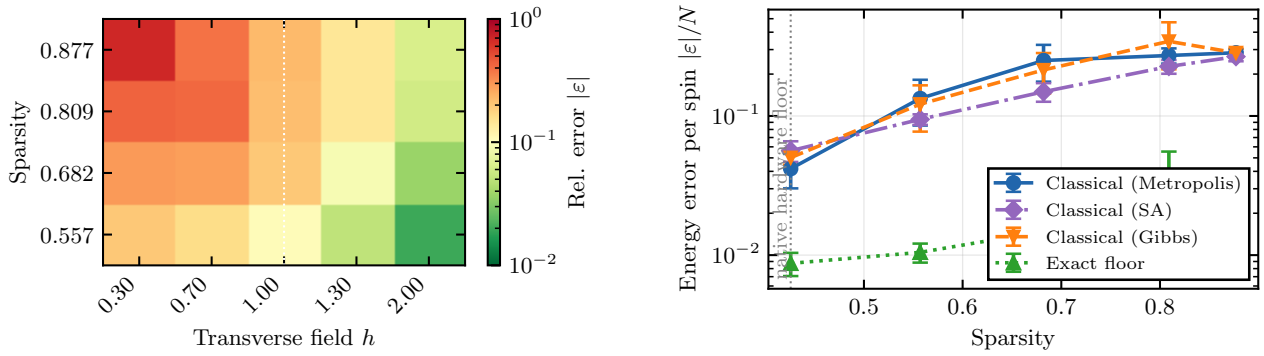


Figure 13: Construction of a chain-free sparse RBM from a D-Wave Zephyr hardware graph ($N = 8$ visible units). **(a)** Identify internal couplers (green) and external/odd couplers (purple)**(b)** Creation of bipartite graph by restricting to internal couplers, splitting into hidden unit candidates (red) and visible units (blue). **(c)** RBM's visible-hidden connectivity mask by selecting the hidden units which maximize connectivity while keeping a min-set-cover. Non-selected candidates (grey) are dropped.

To disentangle how much of this degradation is a fundamental limit of the sparse ansatz itself versus an artefact of MCMC training, we additionally trained the same four masks ($N = 16$, $h = 1$, zephyr, $\alpha = 1$) via exact enumeration of all 2^{16} visible configurations weighted by the exact $|\Psi(v)|^2$ probability, removing all sampling noise and MCMC bias from the optimization. This exact-ansatz floor's mean energy error per spin, $|\varepsilon|/N$ (the quantity plotted in Figure 14b), is 1.05%, 1.48%, 3.45%, and 2.41% across the four masks (sparsity 0.557 to 0.877) — it roughly triples before falling back, rather than staying flat, and peaks at the third mask — in contrast to the classical-MCMC curve, whose mean per-spin error rises from 13.4% to 28.5% over the same range, i.e. 8 to 17 times larger than the floor depending on the mask. The spread across seeds also grows sharply with sparsity, while the floor's own seed-to-seed spread stays comparatively small at low sparsity. This indicates that most of the degradation visible in Figure 14 is driven by optimization/sampling difficulty rather than by the sparse ansatz running out of representational power. This ablation was run at a single system size ($N = 16$) and one specific hardware subgraph per sparsity level, so it should be read as an indicator, and not as a general claim about all sparsity regimes. We conclude that a native Pegasus/Zephyr embedding is



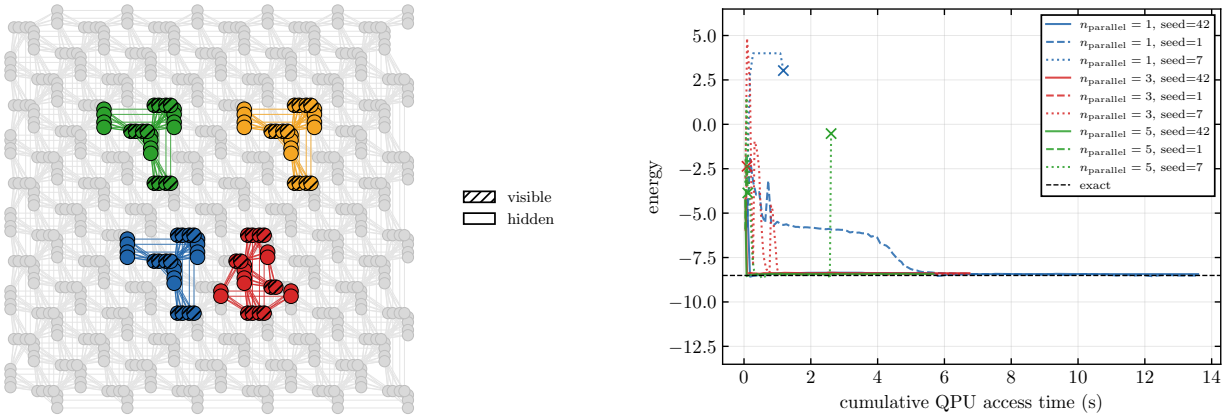
(a) (b)

Figure 14: Error as a function of sparsity, with the number of hidden units kept equal to the number of visible units in both plots. **(a)** Relative error $|\varepsilon|/|E_{\text{exact}}|$ for different values of h, trained classically via Metropolis sampling. **(b)** Energy error per spin, $|\varepsilon|/N$, at $h = 1$ across three classical samplers (Metropolis, simulated annealing, and persistent Gibbs), all using identical hyperparameters (300 SR iterations, learning rate 0.05), against the exact-ansatz floor, which uses noise-free gradients for up to 3000 iterations to escape optimization plateaus. The dotted line marks the exact-ansatz floor at this size. The vertical dashed grey line marks the native hardware floor: the sparsity of the unpruned chain-free embedding for this system size, i.e. the fraction of visible-hidden pairs with no real coupler between them before any edges are deliberately removed. All tested sparsity levels lie to its right, since pruning can only remove couplers, never add ones the chip does not have.

not suitable for efficient RBM sampling, for the following reasons. Even in the optimal case, representability of the sparse ansatz is limited: the noise-free, exact-enumeration floor more than triples as native couplers are pruned away (1.05% to 3.45% per-spin error), showing that sparsification costs real representational capacity independent of how the model is trained. In actual training, effects such as finite-sample estimator noise and MCMC/annealing mixing behavior further degrade performance substantially: across three independent classical samplers (Metropolis-Hastings, simulated annealing, and persistent-chain Gibbs), trained under identical hyperparameters and iteration budgets, the per-spin error at every tested sparsity level sits 8–17× above this floor, with no sampler avoiding the gap. For budget constraints, we tested the sparse models only on classical hardware.

4.5.2 Parallel Embeddings

To speed up calculation and save computing time, it is possible to embed multiple smaller, independent problems on the QPU. For example, it is possible to gather 1000 independent samples using only 250 annealing runs, by embedding the RBM four times on the QPU. We experimentally validate this approach with 3 seeds per $n_{\text{parallel}} \in \{1, 3, 5\}$, shown individually rather than aggregated to a single "best" run per level, as Figure 15 makes visible that training at this iteration budget is not reliable: 4 of the 9 runs diverge (crossed-off markers), including 2 of 3 seeds at $n_{\text{parallel}} = 5$. Among the runs that do converge, higher n_{parallel} still reaches the same energy plateau using markedly less cumulative QPU time (e.g. 13.6 s at $n_{\text{parallel}} = 1$ versus 5.7 s at $n_{\text{parallel}} = 5$), so the core speedup claim holds – but with only 3 seeds per level, the divergence rate itself should be read as suggestive, not a precise reliability estimate.



(a) Four independent $K_{8,8}$ RBM embeddings ($n_{\text{parallel}} = 4$) placed simultaneously on the Pegasus chip. Colors distinguish copy 1-4; hatched markers denote visible units, solid markers denote hidden units.

(b) Energy vs. cumulative QPU access time for $n_{\text{parallel}} \in \{1, 3, 5\}$, all 3 seeds shown individually. $\times$ markers indicate a run diverged before reaching the plateau; among the seeds that converge, higher n_{parallel} reaches the plateau using less real QPU time.

Figure 15: Parallel embedding on a D-Wave QPU: (a) physical embedding layout, (b) resulting training speedup.

5 Implementation

In this section, we are going to describe the conceptual and technological aspects of implementing variational algorithms in a hybrid quantum / classical setup.

5.1 Stochastic gradient descent versus Stochastic reconfiguration

Traditionally, neural networks are trained using gradient descent. We will not go into details about this well known approach, as it is extensively covered by literature. Gradients are computed efficiently using backpropagation which exploits the chain rule, reusing intermediate derivatives to evaluate the compositions of continuous functions. For most of this article, we use the stochastic reconfiguration (SR) method introduced in [11]. The main advantage of SR over gradient descent is that it exploits more knowledge about the ansatz than gradient descent. In particular, gradient descent lives in the *parameter space*, and does not take into account the influence of parameter changes on the actual wave function *probability space* $|\Psi|^2$.

5.1.1 Stochastic reconfiguration

SR is implemented as follows: given the logarithmic derivative for each parameter k, that is the gradient observable

$$O_k(\mathbf{v}) = \frac{\partial \log \Psi(\mathbf{v})}{\partial \theta_k}, \tag{17}$$

with $\boldsymbol{\theta} = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$, the energy gradient or force vector is defined as

$$F_k = \langle O_k E_{\text{loc}} \rangle - \langle O_k \rangle \langle E_{\text{loc}} \rangle, \tag{18}$$

which is the covariance of the gradient observable and the local energy. The local energy $E_{\text{loc}}(\mathbf{v})$ estimates the variational energy $\langle H \rangle$ of the current RBM ansatz using samples drawn from $|\Psi(\mathbf{v})|^2$:

$$E_{\text{loc}}(\mathbf{v}) = \frac{\langle \mathbf{v}|H|\Psi\rangle}{\langle \mathbf{v}|\Psi\rangle} = \sum_{\mathbf{v}'} H_{\mathbf{v}\mathbf{v}'} \frac{\Psi(\mathbf{v}')}{\Psi(\mathbf{v})} \tag{19}$$

The next step then is to define the quantum geometric tensor $\mathbf{S}$, the regularised sample covariance of the O_k vectors, which measures how sensitive the Born distribution $|\Psi|^2$ is to each parameter direction. In matrix form, if $\bar{\mathbf{O}}$ is the matrix of mean-centred gradients, then $\mathbf{S} = \frac{1}{N_s} \bar{\mathbf{O}}^\top \bar{\mathbf{O}} + \lambda I$, with a small regularisation λ added for numerical stability. After solving the SR system

$$\mathbf{S} \cdot \delta\boldsymbol{\theta} = \mathbf{F} \tag{20}$$

for $\delta\boldsymbol{\theta}$, the parameters are updated by

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\, \delta\boldsymbol{\theta}, \tag{21}$$

with learning rate η.

Algorithm 1 Stochastic Reconfiguration

Require: Parameters $\boldsymbol{\theta} = \{\mathbf{a}, \mathbf{b}, \mathbf{W}\}$, learning rate η, regularisation λ

for $t = 1, 2, \ldots$ **do**
  Sample $\mathbf{v}^{(s)} \sim |\Psi_{\boldsymbol{\theta}}|^2$ for $s = 1, \ldots, N_s$ ▷ MCMC
  $E_{\mathrm{loc}}(\mathbf{v}^{(s)}) \leftarrow \sum_{\mathbf{v}'} H_{\mathbf{v}\mathbf{v}'}\, \Psi(\mathbf{v}')/\Psi(\mathbf{v})$ for each s
  $O_k(\mathbf{v}^{(s)}) \leftarrow \partial \log \Psi(\mathbf{v}^{(s)})/\partial\theta_k$ for each s, k
  $F_k \leftarrow \langle O_k E_{\mathrm{loc}}\rangle - \langle O_k\rangle\langle E_{\mathrm{loc}}\rangle$
  $\mathbf{S} \leftarrow \frac{1}{N_s}\bar{\mathbf{O}}^\top\bar{\mathbf{O}} + \lambda I$
  Solve $\mathbf{S}\,\delta\boldsymbol{\theta} = \mathbf{F}$ ▷ conjugate gradient, matrix-free
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\delta\boldsymbol{\theta}$
end for

5.1.2 Conjugate gradient

As $\mathbf{S} \in \mathbb{R}^{N_{\mathrm{params}} \times N_{\mathrm{params}}}$ grows quadratically with the number of parameters $N_{\mathrm{params}} = N + M + NM$, with N and M the number of visible and hidden units respectively, solving the SR system in Equation 20 is intractable for large system sizes. We therefore apply a technique called conjugate gradient (CG), which requires only matrix vector products $\mathbf{S} \cdot \mathbf{x}$ and converges for any symmetric positive-definite matrix. Computing the matrix–vector product then costs only $\mathcal{O}(N_s \cdot N_{\mathrm{params}})$:

$$\mathbf{S} \cdot \mathbf{x} = \frac{1}{N_s}\bar{\mathbf{O}}^\top(\bar{\mathbf{O}}\,\mathbf{x}) + \lambda\mathbf{x} \tag{22}$$

5.2 Exact Solving

Determining the exact value of the ground state up to numerical precision is crucial for evaluating the performance of the trained models. For TFIM, exact solving is possible for arbitrary system sizes, whereas solving the J_1-J_2 Heisenberg model is computationally hard.

5.2.1 TFIM ground state solving in $O(N)$

The 1D TFIM defined in Equation 2 can be solved exactly in $O(N)$ time, for $J > 0$, $h \geq 0$, and periodic boundary conditions on the spin chain – the regime used throughout this work. Dziarmaga [12] solves the identical Hamiltonian by mapping it to a system of free fermions via the Jordan–Wigner transformation, followed by a Fourier and Bogoliubov transformation. The periodic boundary conditions split the fermionic Hilbert space into two parity sectors, each restricted to its own set of allowed momenta; for $J > 0$, $h \geq 0$ the ground state always sits in the antiperiodic sector. Directly applying the quasiparticle dispersion of Eq. (16) and the ground-state (vacuum) energy of Eq. (18) in [12] (denoted $\omega(k)$ here to avoid colliding with the TTE target ϵ used elsewhere in this work; with $g = h/J$; see also [13] for the parity-sector momenta) gives the closed-form ground-state energy:

$$E_0 = -\sum_{m=0}^{N-1} \omega\left(\frac{\pi(2m+1)}{N}\right), \qquad \omega(k) = \sqrt{J^2 + h^2 - 2Jh\cos(k)}. \tag{23}$$

We verified this closed-form result numerically against exact diagonalization (`netket`, using `scipy`'s sparse `eigsh`) for $N \in \{4, 6, 8, 10\}$ and $h \in \{0.5, 1.0, 1.5\}$, finding agreement to machine precision ($< 10^{-13}$).

5.2.2 Exact Diagonalization

In cases where mapping to a quadratic fermion Hamiltonian is not possible, e.g. in the case of long range interactions, exact diagonalization is used for small system sizes N. Exact diagonalization works by enumerating all 2^N basis states of the Hilbert space, constructing the Hamiltonian as a $2^N \times 2^N$ matrix in that basis, and finding its lowest eigenvalue via the Lanczos algorithm. The Lanczos algorithm avoids diagonalizing the entire matrix and only returns the matrix's lowest eigenvalue. We use `scipy`'s `eigsh` implementation of this algorithm.

5.2.3 Other methods

The variational principle bounds the exact expectation value $\langle H\rangle_\Psi = \langle\Psi_{\boldsymbol{\theta}}|H|\Psi_{\boldsymbol{\theta}}\rangle/\langle\Psi_{\boldsymbol{\theta}}|\Psi_{\boldsymbol{\theta}}\rangle$ for a fixed parameter vector $\boldsymbol{\theta}$: it can never fall below the true ground-state energy.

5.3 JAX

(Quantum) variational algorithms do not only strain *quantum* resources, but also push *classical* computers to their limits. Notably, at each iteration, the stochastic reconfiguration process described in subsection 5.1 adapts all parameters of the weights and biases. We use several techniques in this project to speed up this process. JAX, implementing the *functional programming* paradigm, allows for the speed up of function calls and linear algebra calculation by offering features such as just-in-time (JIT) compilation and XLA (accelerated linear algebra). We will describe the features of JAX in detail and give examples from our codebase.

5.3.1 Just-in-time compilation

JAX transformations require functions to be *functionally pure*. To summarize, a function is considered functionally pure, if its output depends only on the function's input parameters, and all output is passed through the return variable. As an example, a function that involves `print()` statements or writes to global variables is not considered to be functionally pure, as its output does not exclusively pass through the `return` statement. JAX does not guarantee the proper behavior of impure functions. During JIT compilation, the function is transformed to primitives, which in JAX are defined to be "fundamental units of computation"[14]. As an example, the following function header is used to calculate the local energy (Equation 19):

```
@functools.partial(jax.jit, static_argnums=(4, 5))
def _local_energy_1d_jit(
    V: jax.Array,  # (ns, N) -- spin configs
    W: jax.Array,  # (N, M)  -- RBM weights
    a: jax.Array,  # (N,)    -- visible biases
    b: jax.Array,  # (M,)    -- hidden biases
    h: float,      # static
    N: int,        # static
) -> jax.Array:    # (ns,)   local energies
    ...
```

h and N are passed as static parameters, meaning that they will be hard-coded in the compiled function after JIT. JAX compiles a kernel for each combination of static variables; as h and N never change during a training run, this means that the function is only compiled once.

5.3.2 Vectorized Map

`jax.vmap` takes a function written for a single input and returns a new function that maps it over a leading batch axis as a single vectorised kernel.

As a concrete example, consider `log_psi_fn` defined for a single spin configuration $\mathbf{v} \in \{-1, +1\}^N$. Evaluating it over all n_s sampled configurations reduces to:

```
log_p_V = jax.vmap(log_psi_fn)(V)    # V: (ns, N) -> log_p_V: (ns,)
```

XLA compiles this as a single operation over the batch dimension rather than n_s independent kernel launches. The resulting operation can be compiled using JIT for faster execution:

```
batched_f = jax.jit(jax.vmap(log_psi_fn))
```

5.3.3 Autograd

JAX offers automatic differentiation to compute function gradients. However, we do not make use of this feature as gradients are calculated analytically (see [15]).

5.4 Hardware and Software Configuration

D-Wave experiments use the `Advantage_system6` (Pegasus) and `Advantage2_system1` (Zephyr) solvers. Forward-anneal runs use a $20\,\mu$s anneal time and `num_reads` equal to the requested sample count, with `auto_scale` enabled (see subsubsection 4.3.2 for why this matters for β_x) and D-Wave's default chain strength unless stated otherwise; embeddings are cached per `(n_visible, solver)` using `minorminer`'s biclique embedder. Classical simulated-annealing baselines use `dwave-neal` with 1000 sweeps and a geometric β schedule over $[0.01, 10]$. FPGA/VeloxQ [8] runs use `Float32` precision, a 100 MHz core clock, 1024 parallel replicas, and 1000 update steps per run. All stochastic components (JAX/NumPy) are seeded per run via `np.random.randint` feeding `jax.random.PRNGKey`.

6 Appendix

Claim. Given the regularised quantum geometric tensor $\mathbf{S} = \frac{1}{N_s}\bar{\mathbf{O}}^\top\bar{\mathbf{O}} + \lambda I$, its product with any vector $\mathbf{x} \in \mathbb{R}^{N_{\text{params}}}$ satisfies

$$\mathbf{S}\,\mathbf{x} = \frac{1}{N_s}\bar{\mathbf{O}}^\top(\bar{\mathbf{O}}\,\mathbf{x}) + \lambda\mathbf{x}. \tag{24}$$

Derivation.

Step 1: Write the unregularised covariance in sample form. Define $\Sigma_{kl} = \langle O_k O_l\rangle - \langle O_k\rangle\langle O_l\rangle$, the unregularised sample covariance of the gradient observables, so that $\mathbf{S} = \mathbf{\Sigma} + \lambda I$. Introducing the matrix $\mathbf{O} \in \mathbb{R}^{N_s \times N_{\text{params}}}$ with entries $O_{sk} = O_k(\mathbf{v}^{(s)})$, the sample estimates are

$$\langle O_k O_l\rangle = \frac{1}{N_s}\sum_s O_{sk}\,O_{sl} = \frac{1}{N_s}(\mathbf{O}^\top\mathbf{O})_{kl}, \qquad \langle O_k\rangle\langle O_l\rangle = \mu_k\mu_l, \tag{25}$$

where $\mu_k = \frac{1}{N_s}\sum_s O_{sk}$ is the sample mean of the k-th observable.

Step 2: Express $\mathbf{\Sigma}$ via the mean-centred matrix. Define $\bar{\mathbf{O}} \in \mathbb{R}^{N_s \times N_{\text{params}}}$ by $\bar{O}_{sk} = O_{sk} - \mu_k$. Then

$$\frac{1}{N_s}(\bar{\mathbf{O}}^\top\bar{\mathbf{O}})_{kl} = \frac{1}{N_s}\sum_s (O_{sk}-\mu_k)(O_{sl}-\mu_l) = \langle O_k O_l\rangle - \mu_k\mu_l = \Sigma_{kl}, \tag{26}$$

so, including regularisation,

$$\mathbf{S} = \mathbf{\Sigma} + \lambda I = \frac{1}{N_s}\bar{\mathbf{O}}^\top\bar{\mathbf{O}} + \lambda I, \tag{27}$$

matching the Claim.

Step 3: Apply to $\mathbf{x}$. Since matrix multiplication is associative,

$$\mathbf{S}\,\mathbf{x} = \frac{1}{N_s}\bar{\mathbf{O}}^\top\bar{\mathbf{O}}\,\mathbf{x} + \lambda I\,\mathbf{x} = \frac{1}{N_s}\bar{\mathbf{O}}^\top\underbrace{(\bar{\mathbf{O}}\,\mathbf{x})}_{\mathbf{z}\,\in\,\mathbb{R}^{N_s}} + \lambda\mathbf{x}. \qquad \square \tag{28}$$

Step 4: Two-pass interpretation. The product $\mathbf{Sx}$ therefore decomposes into two operations:

1. **Forward pass.** Compute $\mathbf{z} = \bar{\mathbf{O}}\,\mathbf{x} \in \mathbb{R}^{N_s}$, where $z_s = \sum_k \bar{O}_{sk}\,x_k$ is the projection of $\mathbf{x}$ onto the centred gradient of sample s. Cost: $\mathcal{O}(N_s \cdot N_{\text{params}})$.
2. **Backward pass.** Compute $\bar{\mathbf{O}}^\top\mathbf{z} \in \mathbb{R}^{N_{\text{params}}}$, where $(\bar{O}^\top z)_k = \sum_s \bar{O}_{sk}\,z_s$ back-projects the per-sample scalars into parameter space. Cost: $\mathcal{O}(N_s \cdot N_{\text{params}})$.

The matrix $\mathbf{S} \in \mathbb{R}^{N_{\text{params}} \times N_{\text{params}}}$ is never formed. Total cost per matrix–vector product: $\mathcal{O}(N_s \cdot N_{\text{params}})$, identical to evaluating the gradient once.

References

[1] Philipp Hanussek, Jakub Pawłowski, Zakaria Mzaouali, and Bartłomiej Gardas. Quantum-inspired dynamical models on quantum and classical annealers. *Phys. Rev. Appl.*, 25:044055, apr 2026.

[2] Konrad Jałowiecki, Andrzej Więckowski, Piotr Gawron, and Bartłomiej Gardas. Parallel in time dynamics with quantum annealers. *Scientific Reports*, 10(1):13534, 2020.

[3] Vrinda Mehta, Hans De Raedt, Kristel Michielsen, and Fengping Jin. Understanding the physics of d-wave annealers: From schrödinger to lindblad to markovian dynamics. *Physical Review A*, 112(3), September 2025.

[4] Jon Nelson, Marc Vuffray, Andrey Y. Lokhov, Tameem Albash, and Carleton Coffrin. High-quality thermal gibbs sampling with quantum annealing hardware. *Physical Review Applied*, 17(4), April 2022.

[5] Max-Planck-Gesellschaft. Quantum magnetism lecture notes. `https://www.pks.mpg.de/fileadmin/user_upload/MPIPKS/group_pages/SMC/quantum_magnetism_lecture_notes/lecture9.pdf`. [Accessed 29-06-2026].

[6] Kentaro Kubo and Hayato Goto. Unlocking the power of boltzmann machines by parallelizable sampler and efficient temperature estimation, 2025.

[7] Danielle Navarro. The Metropolis-Hastings algorithm. `https://blog.djnavarro.net/posts/2023-04-12_metropolis-hastings/#fnref1`. [Accessed 16-06-2026].

[8] J. Pawłowski, J. Tuziemski, P. Tarasiuk, A. Przybysz, R. Adamski, K. Hendzel, Ł. Pawela, and B. Gardas. Veloxq: A fast and efficient qubo solver, 2025.

[9] J. Richter, N. B. Ivanov, and K. Retzlaff. On the violation of Marshall-Peierls sign rule in the frustrated J_1-J_2 Heisenberg antiferromagnet. *Europhysics Letters*, 25(7):545–550, 1994.

[10] Matthias Troyer and Uwe-Jens Wiese. Computational complexity and fundamental limitations to fermionic quantum monte carlo simulations. *Physical Review Letters*, 94(17), May 2005.

[11] Sandro Sorella. Generalized lanczos algorithm for variational quantum monte carlo. *Physical Review B*, 64(2), June 2001.

[12] Jacek Dziarmaga. Dynamics of a quantum phase transition: Exact solution of the quantum ising model. *Physical Review Letters*, 95(24):245701, December 2005.

[13] Yan He and Hao Guo. The boundary effects of transverse field ising model. *Journal of Statistical Mechanics: Theory and Experiment*, 2017(9):093101, September 2017.

[14] JAX Developers. Glossary of terms — JAX documentation — docs.jax.dev. `https://docs.jax.dev/en/latest/glossary.html`, 2026. [Accessed 30-06-2026].

[15] Bartłomiej Gardas, Marek M. Rams, and Jacek Dziarmaga. Quantum neural networks to simulate many-body quantum systems. *Physical Review B*, 98(18), November 2018.